Actividad 3.1 (Trayectorias en lazo abierto)

Ivan Ariel Rebollar León | A01737891

Bruno Alejandro Domínguez Maldonado | A01738844

Omar Kuri Vázquez | A01738129

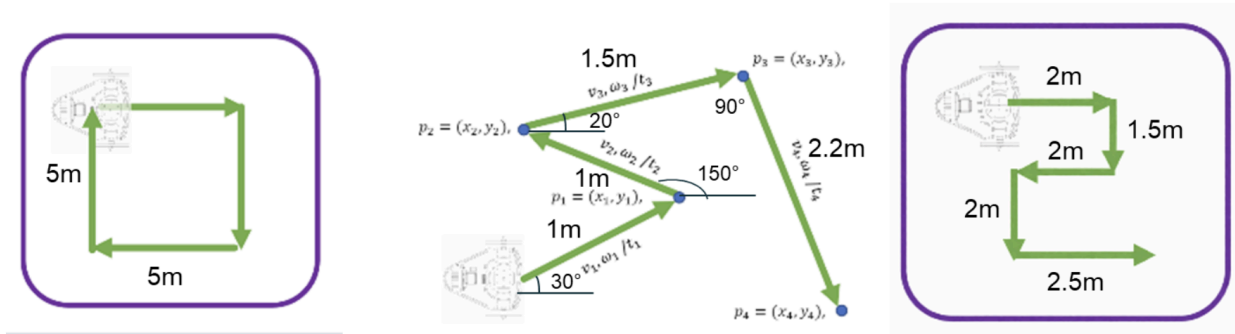
Erik García Cruz | A01732440

8 de Abril del 2026

Fundamentación de robótica (Gpo 101)

Alfredo García Suárez

A) **B)** **C)**



A)

Paso 1: Preparar el entorno y las constantes de tiempo

```
None
clear
close all
clc
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% TIEMPO
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
tf = 24;              % Tiempo de simulacion
ts = 0.1;             % Tiempo de muestreo en segundos (s)
t = 0: ts: tf;        % Vector de tiempo
N = length(t);        % Muestras
```

Aquí:

- Se limpia el espacio de trabajo, se cierran las figuras y se limpia la consola para evitar interferencias de ejecuciones anteriores. Se define el tiempo final de la simulación (tf), el tiempo de muestreo (ts), y se crea un vector de tiempo (t) que permite calcular el número total de muestras (N) que determinarán cuántas iteraciones tendrá el bucle principal.

Paso 2: Establecer las condiciones iniciales y el punto de control

```
None
%%%%%%%%%%%%%%%%%%%%%%% CONDICIONES INICIALES
%%%%%%%%%%%%%%%%%%%%%%%%%%%
x1 = zeros (1,N+1);  % Posición en el centro del eje que
une las ruedas (eje x) en metros (m)
y1 = zeros (1,N+1);  % Posición en el centro del eje que
une las ruedas (eje y) en metros (m)
phi = zeros(1, N+1); % Orientacion del robot en radianes
(rad)
x1(1) = 0;    % Posicion inicial eje x
y1(1) = 0;   % Posicion inicial eje y
phi(1) = pi/2;   % Orientacion inicial
%%%%%%%%%%%%%%%%%%%%%%%%%%% PUNTO DE CONTROL
%%%%%%%%%%%%%%%%%%%%%%%%%%%
hx = zeros(1, N+1);  % Posicion en el punto de control (eje
x) en metros (m)
hy = zeros(1, N+1);  % Posicion en el punto de control (eje
y) en metros (m)
hx(1) = x1(1); % Posicion en el punto de control del robot
en el eje x
hy(1) = y1(1); % Posicion en el punto de control del robot
en el eje y
```

Aquí:

- Se inicializa con ceros los arreglos para las posiciones y la orientación, lo cual es una buena práctica en MATLAB para optimizar la memoria, luego, se establecen los valores iniciales reales: el robot comienza en el origen (0,0) apuntando hacia arriba (orientación de 90 grados) y también se inicializan las variables del punto de control (hx, hy) que seguirán la trayectoria del centro del eje de las ruedas.

Paso 3: Definir las velocidades de referencia para trazar un cuadrado

```
None
%%%%%%%%%%%%%%%%%%%%% VELOCIDADES DE REFERENCIA
%%%%%%%%%%%%%%%%%%%%%%%%%
u = zeros(1,N); % Inicializamos vector de velocidad lineal
w = zeros(1,N); % Inicializamos vector de velocidad angular
% Definición de tramos para el cuadrado
% Tramo 1
u(1:50) = 1;
% Giro 1
w(51:60) = -(pi/2);
% Tramo 2
u(61:110) = 1;
% Giro 2
w(111:120) = -(pi/2);
% Tramo 3
u(121:170) = 1;
% Giro 3
w(171:180) = -(pi/2);
% Tramo 4
u(181:230) = 1;
```

```
% Giro 4
w(231:240) = -(pi/2);
```

Aquí:

- Se preasignan los vectores de las señales de control de velocidad lineal (u) y angular (w). Se diseña la trayectoria dividiendo las muestras en secuencias específicas: primero el robot avanza en línea recta (velocidad lineal activa, velocidad angular nula), y luego gira sobre su propio eje (velocidad lineal nula, velocidad angular negativa - π/2 rad/s) para hacer esquinas de 90 grados. Esto se repite cuatro veces para completar la figura cuadrada.

Paso 4: Ejecutar el bucle de simulación usando el modelo cinemático

```
None
%%%%%%%%%%%%%%%%%%%%%%%% BUCLE DE SIMULACION
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
for k=1:N
   phi(k+1)=phi(k)+w(k)*ts; % Integral numérica (método de
Euler)

   %%%%%%%%%%%%%%%%%%%% MODELO CINEMATICO
%%%%%%%%%%%%%%%%%%%%%%%%%
   %Aplicamos el modelo cinemático diferencial para obtener
las
   %velocidades en x, y, phi
   xp1=u(k)*cos(phi(k+1));
```

```
   yp1=u(k)*sin(phi(k+1));
   phip= w(k);
   x1(k+1)=x1(k) + xp1*ts ; % Integral numérica (método de
Euler)
   y1(k+1)=y1(k) + yp1*ts ; % Integral numérica (método de
Euler)

   % Posicion del robot con respecto al punto de control
   hx(k+1)=x1(k+1);
   hy(k+1)=y1(k+1);
end
```

Aquí:

- Se iteran todas las muestras de tiempo. En cada paso, se utiliza el método de integración numérica de Euler para actualizar la posición y orientación del robot con base en el modelo cinemático diferencial de un robot tipo uniciclo, primero se actualiza la orientación (phi), luego se descomponen las velocidades en los ejes cartesianos (xp1, yp1), y finalmente se integran para obtener la nueva posición (x1, y1) y se guardan en el historial del punto de control.

Paso 5: Configurar y ejecutar la animación de la simulación 3D

```
None
%%%%%%%%%%%%%%%%%%%%%%%%%%%%% SIMULACION VIRTUAL 3D
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% a) Configuracion de escena
scene=figure;  % Crear figura (Escena)
set(scene,'Color','white'); % Color del fondo de la escena
set(gca,'FontWeight','bold') ;% Negrilla en los ejes y
etiquetas
sizeScreen=get(0,'ScreenSize'); % Retorna el tamaño de la
pantalla del computador
set(scene,'position',sizeScreen); % Congigurar tamaño de la
figura
camlight('headlight'); % Luz para la escena
axis equal; grid on; box on;
xlabel('x(m)'); ylabel('y(m)'); zlabel('z(m)'); % Etiqueta
de los eje
view([25 25]); % Orientacion de la figura
axis([-2 7 -2 7 0 2]);
% b) Graficar robots en la posicion inicial
scale = 4;
MobileRobot_5;
H1=MobilePlot_4(x1(1),y1(1),phi(1),scale);hold on;
% c) Graficar Trayectorias
H2=plot3(hx(1),hy(1),0,'r','lineWidth',2);
% d) Bucle de simulacion de movimiento del robot
step=1; % pasos para simulacion
for k=1:step:N
   delete(H1);
```

```
    delete(H2);
    H1=MobilePlot_4(x1(k),y1(k),phi(k),scale);
    H2=plot3(hx(1:k),hy(1:k),zeros(1,k),'r','lineWidth',2);
    pause(ts);
end
```

Aquí:

- Se crea un entorno visual para observar el comportamiento del robot y se configura una ventana adaptada a la pantalla con iluminación y proporciones iguales para los ejes, se llama a funciones externas (MobileRobot_5 y MobilePlot_4) para dibujar el chasis del robot en su posición inicial, mediante un bucle, se redibuja continuamente la posición del robot y el trazo rojo de la trayectoria (H2), borrando la figura anterior y pausando por el tiempo de muestreo (ts) para generar el efecto de animación.

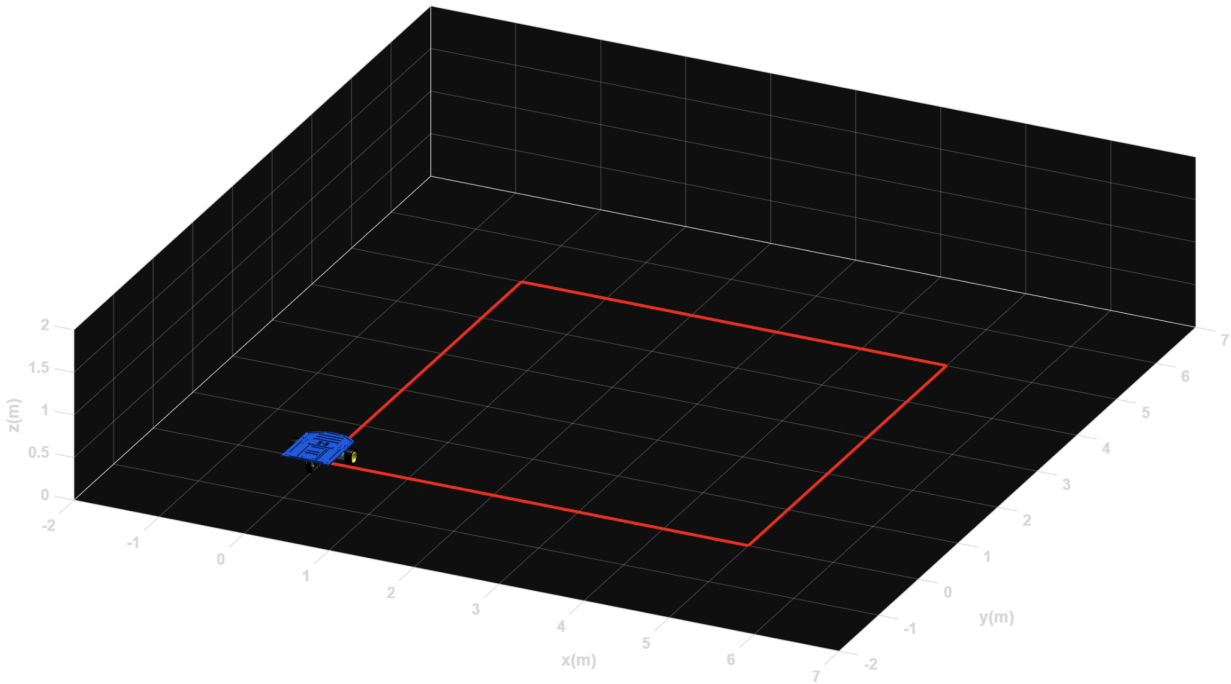
Paso 6: Graficar las señales de control de velocidad

```
None
%%%%%%%%%%%%%%%%%%%%%%%%%%%% Graficas
%%%%%%%%%%%%%%%%%%%%%%%%%%%%
graph=figure;  % Crear figura (Escena)
set(graph,'position',sizeScreen); % Congigurar tamaño de la
figura
subplot(211)
plot(t,u,'b','LineWidth',2),grid('on'),xlabel('Tiempo
[s]'),ylabel('m/s'),legend('u');
subplot(212)
```

```
plot(t,w,'r','LineWidth',2),grid('on'),xlabel('Tiempo
[s]'),ylabel('[rad/s]'),legend('w');
```

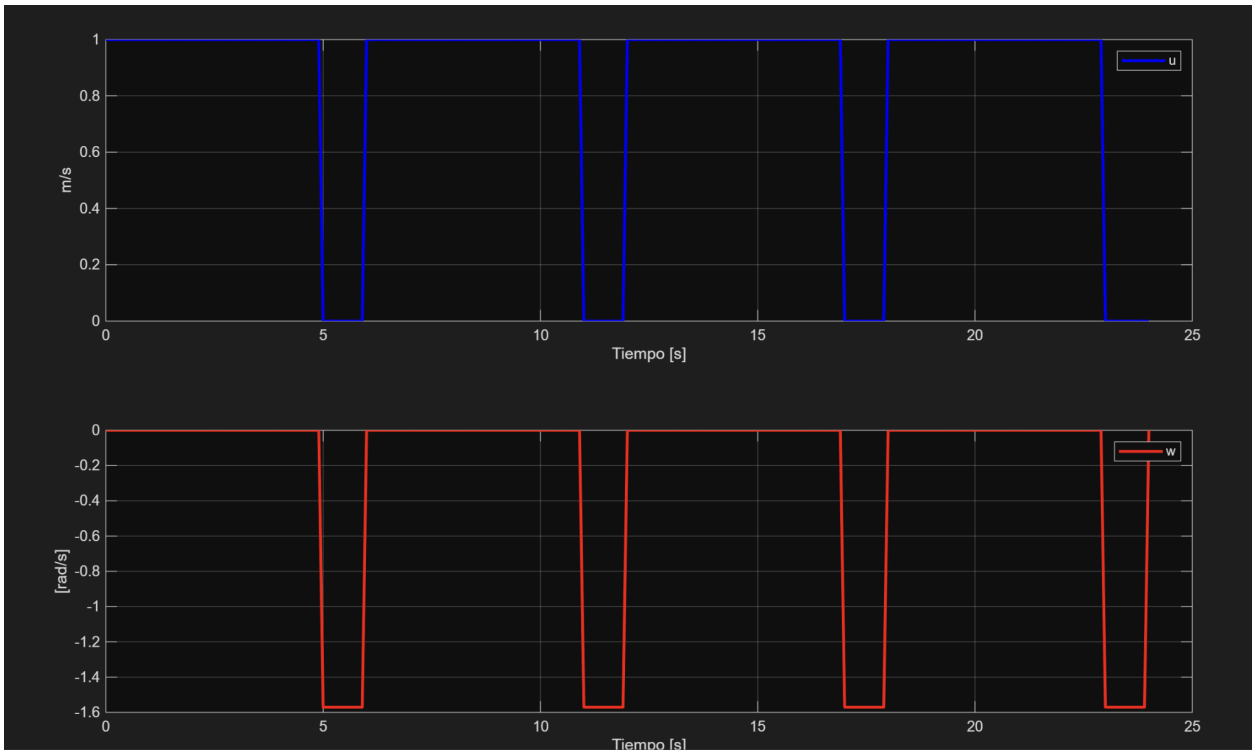
Aquí:

- Se abre una nueva figura para presentar el comportamiento de las entradas del sistema a lo largo del tiempo. Se utilizan subgráficos (subplot) para colocar en la parte superior el perfil de la velocidad lineal (u) y en la parte inferior el perfil de la velocidad angular (w).



La trayectoria del robot se traza con una línea roja continua que marca la huella de su punto de control para formar un cuadrado.

- Avanza en línea recta.
- Gira 90° a la derecha y avanza.
- Gira 90° a la derecha y avanza.
- Gira 90° a la derecha y avanza.
- Gira 90° a la derecha y da el último avance para cerrar la figura.



Gráfica Superior (Línea Azul): Velocidad Lineal (u)

- Muestra la velocidad de avance del robot en metros por segundo (m/s).
- Alterna como una señal de pulsos entre 1 m/s (cuando el robot avanza en línea recta) y 0 m/s (cuando el robot se detiene para girar). Presenta 4 bloques de avance continuo.

Gráfica Inferior (Línea Roja): Velocidad Angular (w)

- Muestra la velocidad de rotación del robot en radianes por segundo (rad/s).
- Se mantiene en 0 rad/s mientras el robot avanza en línea recta (velocidad lineal en 1).

- Presenta cuatro pulsos negativos que llegan a -1.57 rad/s (aproximadamente -pi/2) y esto corresponde a los momentos exactos en que el robot se detiene para girar hacia la derecha y formar las esquinas del cuadrado.

B)

Paso 1: Preparar el entorno y las constantes de tiempo

```
None
clear
close all
clc
%%%%%%%%%%%%%%%%%%%%%%%%%%%%% TIEMPO
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
tf = 10;              % Tiempo de simulacion
ts = 0.1;             % Tiempo de muestreo en segundos (s)
t = 0: ts: tf;        % Vector de tiempo
N = length(t);        % Muestras
```

Aquí:

- Se limpia el espacio de trabajo para asegurar que no existan interferencias de simulaciones previas.
- Se define el tiempo de muestreo (ts) de 0.1 segundos y un tiempo final más corto (tf = 10 s), generando un vector temporal que dictará la duración de cada segmento del movimiento del robot.

Paso 2: Definir condiciones iniciales y punto de control

```
None
%%%%%%%%%%%%%%%%%%%%%%% CONDICIONES INICIALES
%%%%%%%%%%%%%%%%%%%%%%%%%%%%
x1 = zeros (1,N+1);  y1 = zeros (1,N+1);  phi = zeros(1,
N+1);
x1(1) = 0;    y1(1) = 0;   phi(1) = 30*pi/180;
%%%%%%%%%%%%%%%%%%%%%%%%%%% PUNTO DE CONTROL
%%%%%%%%%%%%%%%%%%%%%%%%%%%
hx = zeros(1, N+1);  hy = zeros(1, N+1);
hx(1) = x1(1); hy(1) = y1(1);
```

Aquí:

- Se inicializan los vectores de estado (posición x, y, además de la orientación).
- Se establecen las coordenadas de partida en el origen (0,0), pero con una orientación inicial inclinada de 30 grados (30*pi/180 rad), y se define el punto de control (hx, hy).

Paso 3: Definir las velocidades de referencia (Perfil de movimiento)

```
None
%%%%%%%%%%%%%%%%%%%%%% VELOCIDADES DE REFERENCIA
%%%%%%%%%%%%%%%%%%%%%%%%%%
u = zeros(1,N); w = zeros(1,N);
u(1:10) = 1;                    % Tramo 1
w(11:20) = ((120*pi)/180);      % Giro 1
u(21:30) = 1;                   % Tramo 2
w(31:40) = -(13*pi/18);         % Giro 2
```

```
u(41:50) = 1.5;                    % Tramo 3
w(51:60) = -(pi/2);                % Giro 3
u(61:70) = 2.2;                    % Tramo 4
```

Aquí:

- Se construyen vectores que contienen los comandos de velocidad para cada instante.
- El robot alterna entre avances en línea recta cada vez más rápidos (u = 1, luego u = 1.5, y al final u = 2.2) y diferentes velocidades angulares para ejecutar giros asimétricos específicos que componen su nueva trayectoria.

Paso 4: Ejecutar el bucle de simulación (Modelo Cinemático)

```
None
%%%%%%%%%%%%%%%%%%%%%%%% BUCLE DE SIMULACION
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
for k=1:N
   phi(k+1)=phi(k)+w(k)*ts;
   xp1=u(k)*cos(phi(k+1));
   yp1=u(k)*sin(phi(k+1));
   x1(k+1)=x1(k) + xp1*ts ;
   y1(k+1)=y1(k) + yp1*ts ;
   hx(k+1)=x1(k+1);
   hy(k+1)=y1(k+1);
end
```

Aquí:

- Se utiliza el Método de Euler para integrar las velocidades y obtener la posición actual.
- Primero se actualiza la orientación (phi) y luego se descomponen las velocidades en los ejes cartesianos mediante funciones trigonométricas para calcular el nuevo desplazamiento en x y y.

Paso 5: Visualización animada en 3D

```
None
% d) Bucle de simulacion de movimiento del robot
step=1; % pasos para simulacion
for k=1:step:N
   delete(H1);
   delete(H2);
   H1=MobilePlot_4(x1(k),y1(k),phi(k),scale);
   H2=plot3(hx(1:k),hy(1:k),zeros(1,k),'r','lineWidth',2);
   pause(ts);
end
```

Aquí:

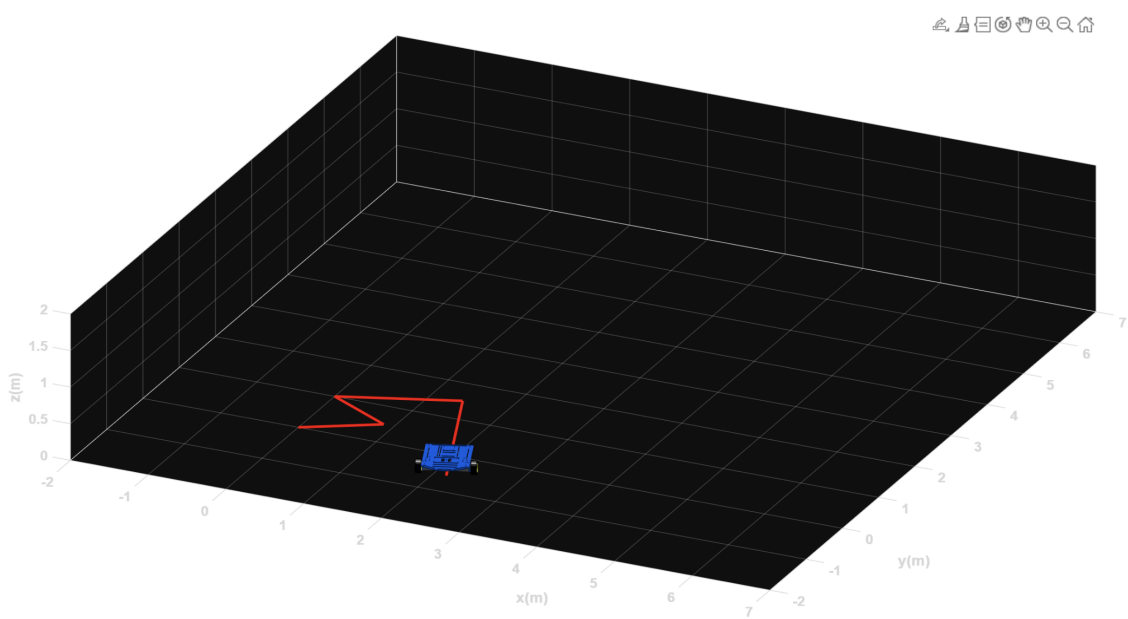
- Se crea un bucle de animación donde se dibuja el robot (H1) y la trayectoria recorrida (H2) en cada paso de tiempo.
- El comando delete elimina la posición anterior del robot para dar la sensación de movimiento fluido, mientras que el plot3 va acumulando los puntos para mostrar el rastro rojo de la trayectoria.

Paso 6: Análisis de señales (Gráficas de velocidad)

```
None
%%%%%%%%%%%%%%%%%%%%%%%%%%%% Graficas
%%%%%%%%%%%%%%%%%%%%%%%%%%%%
subplot(211)
plot(t,u,'b','LineWidth',2),grid('on'),xlabel('Tiempo
[s]'),ylabel('m/s'),legend('u');
subplot(212)
plot(t,w,'r','LineWidth',2),grid('on'),xlabel('Tiempo
[s]'),ylabel('[rad/s]'),legend('w');
```

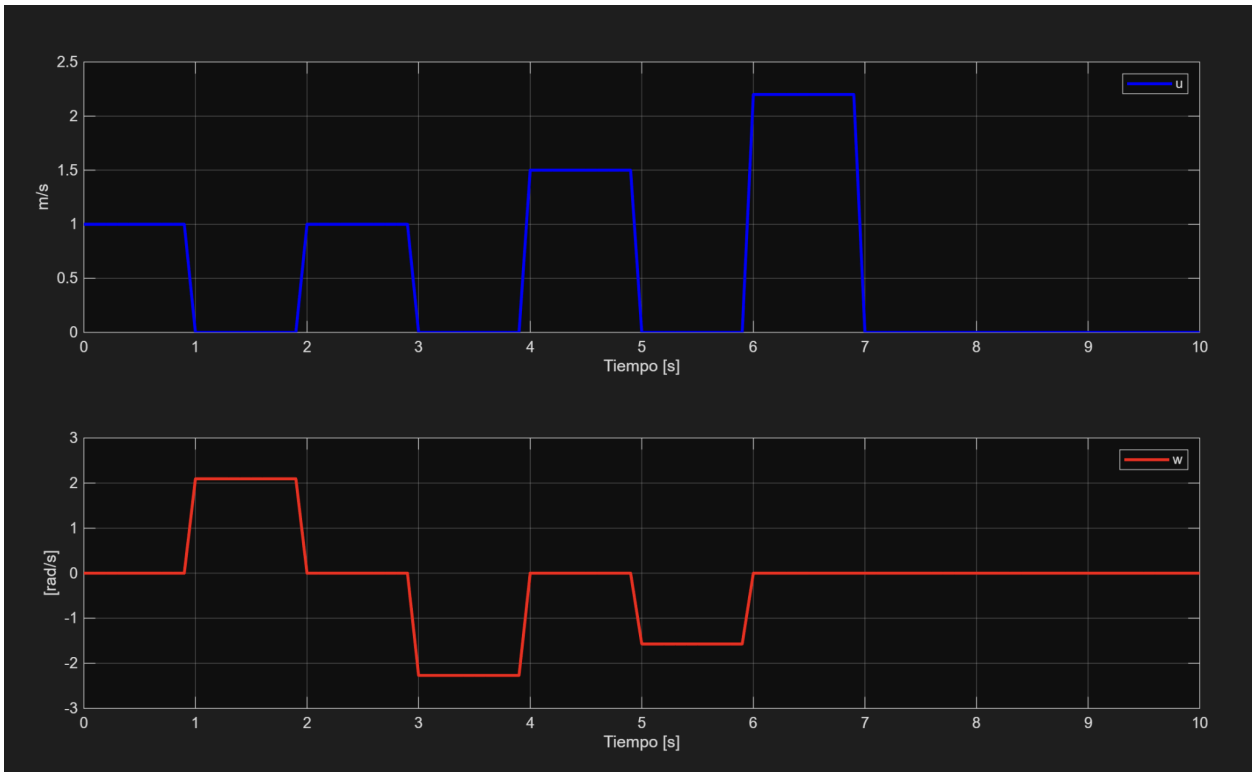
Aquí:

- Se generan sub gráficas para visualizar los perfiles de entrada. Esto permite verificar que el robot se detiene linealmente mientras gira y viceversa, garantizando que el comportamiento físico coincide con la trayectoria geométrica observada en la animación.



La trayectoria del robot se traza con una línea roja continua que marca la huella de su punto de control.

- Comienza apuntando con un ángulo de 30° y avanza en línea recta.
- Gira 120° a la izquierda y avanza.
- Gira 130° a la derecha y avanza de forma más rápida.
- Gira 90° a la derecha y da el último avance aún más veloz.



Gráfica Superior (Línea Azul): Velocidad Lineal (u)

- Muestra la velocidad de avance del robot en metros por segundo (m/s).
- Alterna como una señal de escalones incrementales: empieza con pulsos de 1 m/s, sube a un pulso de 1.5 m/s, y termina con un último pulso de 2.2 m/s (cuando el robot avanza en línea recta). La velocidad cae a 0 m/s en los intervalos donde el robot se detiene para girar.

Gráfica Inferior (Línea Roja): Velocidad Angular (w)

- Muestra la velocidad de rotación del robot en radianes por segundo (rad/s).

- Se mantiene en 0 rad/s mientras el robot avanza en línea recta (cuando la velocidad lineal está activa).
- Presenta un pulso positivo que llega a approx 2.09 rad/s (correspondiente a 120°) indicando un giro hacia la izquierda.
- Presenta un pulso negativo que llega a approx -2.26 rad/s (correspondiente a -13pi/18 o -130°) indicando un giro a la derecha.
- Presenta un último pulso negativo que llega a -1.57 rad/s (aproximadamente -pi/2) y esto corresponde al último momento en que el robot se detiene para girar a la derecha.

C)

Paso 1: Preparar el entorno y las constantes de tiempo

```
None
clear; close all; clc;
tf = 13.9;
ts = 0.1;
t = 0: ts: tf;
N = length(t);
```

Aquí:

- Se limpia el espacio de trabajo para asegurar que no existan interferencias de simulaciones previas.
- Se define el tiempo de muestreo (t_s) y el tiempo final, generando un vector temporal que dictará la duración de cada segmento del movimiento del robot.

Paso 2: Definir condiciones iniciales y punto de control

```
None
x1 = zeros (1,N+1);  y1 = zeros (1,N+1);  phi = zeros(1,
N+1);
x1(1) = 0; y1(1) = 6; phi(1) = 0;
```

```
hx = zeros(1, N+1); hy = zeros(1, N+1);
hx(1) = x1(1); hy(1) = y1(1);
```

Aquí:

- Se inicializan los vectores de estado (posición x, y, ademas de la orientación ϕ)
- Se establecen las coordenadas de partida y se define el punto de control (hx, hy),

Paso 3: Definir las velocidades de referencia (Perfil de movimiento)

```
None
u = [ones(1,20) zeros(1,10) ones(1,15) zeros(1,10) ...];
w = [zeros(1,20) -pi/2*ones(1,10) zeros(1,15)
-pi/2*ones(1,10) ...];
```

Aquí:

- Se construyen vectores que contienen los comandos de velocidad para cada instante.
- El robot alterna entre velocidad lineal constante (u = 1) para avanzar en línea recta y velocidad angular (w = $\pm$ π/2) para ejecutar los giros de 90° que componen la trayectoria.

Paso 4: Ejecutar el bucle de simulación (Modelo Cinemático)

```
None
for k=1:N
    phi(k+1) = phi(k) + w(k)*ts;
    xp1 = u(k)*cos(phi(k+1));
    yp1 = u(k)*sin(phi(k+1));
    x1(k+1) = x1(k) + xp1*ts;
    y1(k+1) = y1(k) + yp1*ts;
end
```

Aquí:

- Se utiliza el Método de Euler para integrar las velocidades y obtener la posición actual.
- Primero se actualiza la orientación (ϕ) y luego se descomponen las velocidades en los ejes cartesianos mediante funciones trigonométricas para calcular el nuevo desplazamiento en x y y.

Paso 5: Visualización animada en 3D

```
None
for k=1:step:N
    delete(H1); delete(H2);
    H1 = MobilePlot_4(x1(k), y1(k), phi(k), scale);
    H2 = plot3(hx(1:k), hy(1:k), zeros(1,k), 'r',
'lineWidth', 2);
    pause(ts);
end
```

Aquí:

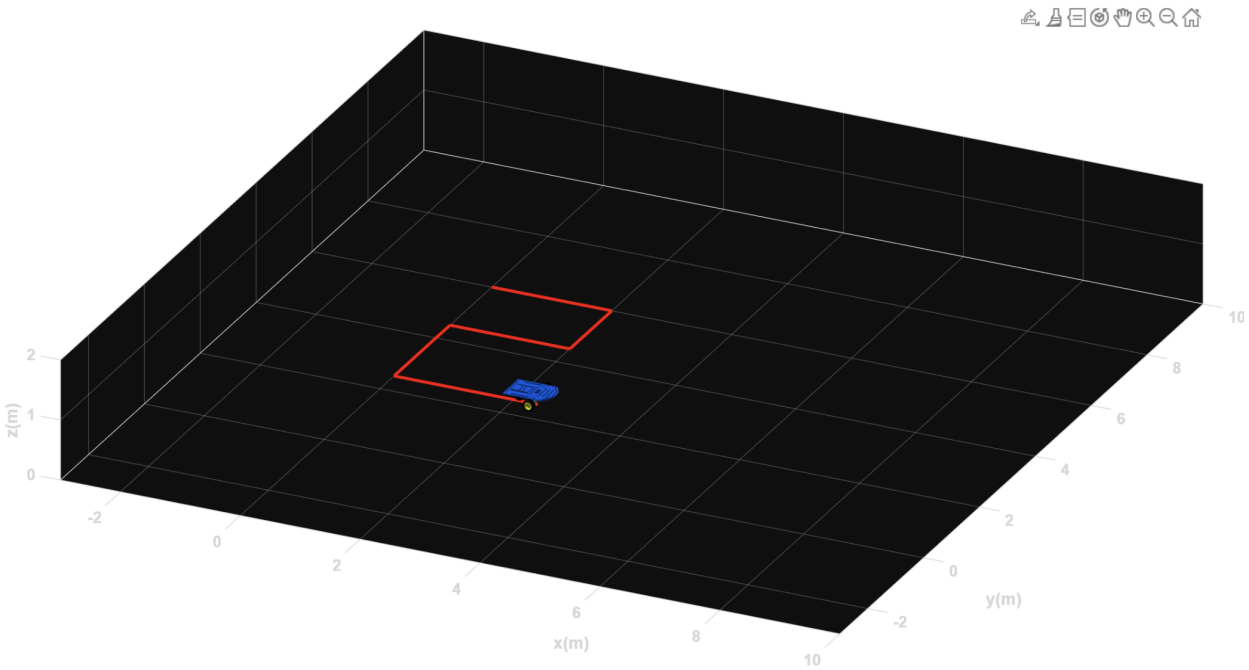
- Se crea un bucle de animación donde se dibuja el robot (H1) y la trayectoria recorrida (H2) en cada paso de tiempo.
- El comando delete elimina la posición anterior del robot para dar la sensación de movimiento fluido, mientras que el plot3 va acumulando los puntos para mostrar el rastro rojo de la trayectoria.

Paso 6: Análisis de señales (Gráficas de velocidad)

```
None
subplot(211)
plot(t, u, 'b', 'LineWidth', 2); title('Velocidad Lineal');
subplot(212)
plot(t, w, 'r', 'LineWidth', 2); title('Velocidad
Angular');
```

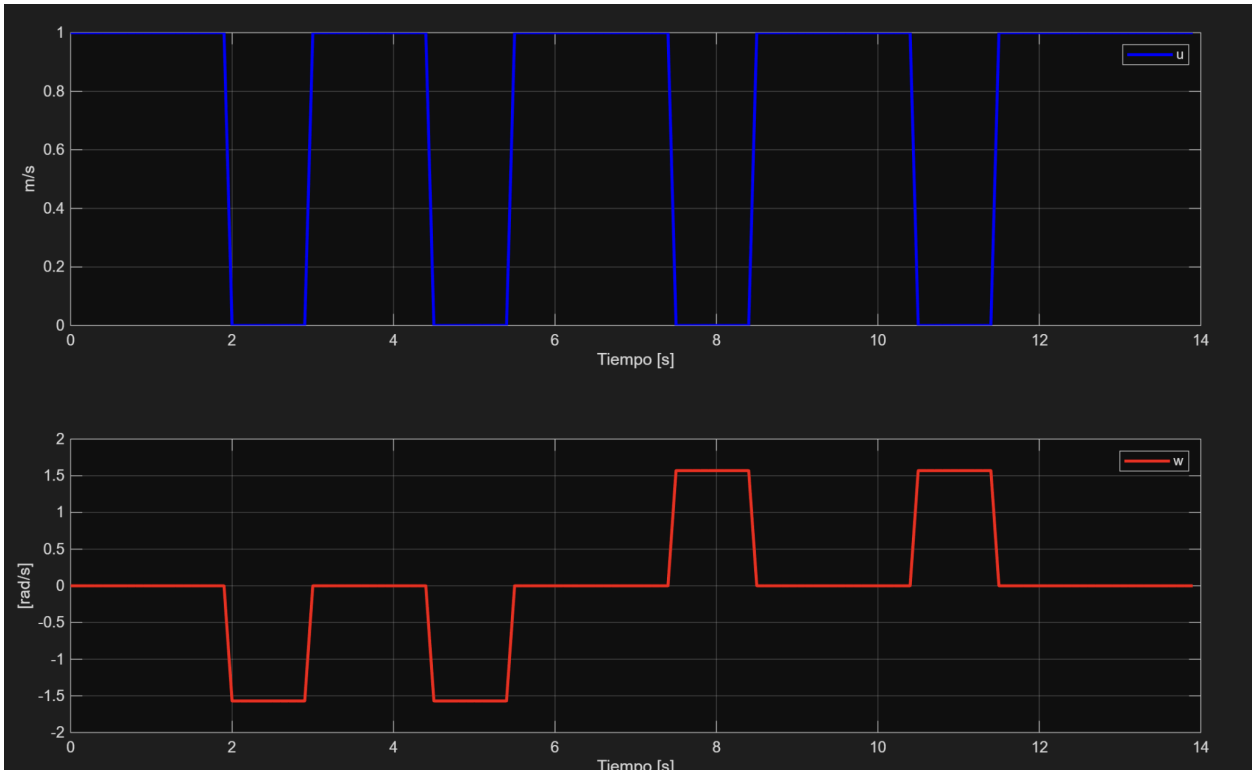
Aquí:

- Se generan subgráficas para visualizar los perfiles de entrada. Esto permite verificar que el robot se detiene linealmente mientras gira y viceversa, garantizando que el comportamiento físico coincide con la trayectoria geométrica observada en la animación



La trayectoria del robot se traza una línea roja continua que marca la huella de su punto de control.

- Avanza en línea recta.
- Gira 90° a la derecha y avanza.
- Gira 90° a la derecha y avanza.
- Gira 90° a la izquierda y avanza.
- Gira 90° a la izquierda y da el último avance.



Gráfica Superior (Línea Azul): Velocidad Lineal (u)

- Muestra la velocidad de avance del robot en metros por segundo (m/s).
- Alterna como una señal de pulsos entre 1 m/s (cuando el robot avanza en línea recta) y 0 m/s (cuando el robot se detiene para girar)

Gráfica Inferior (Línea Roja): Velocidad Angular (w)

- Muestra la velocidad de rotación del robot en radianes por segundo (rad/s).
- Se mantiene en 0 rad/s mientras el robot avanza en línea recta (velocidad lineal en 1).
- Presenta pulsos negativos que llegan a -1.57 rad/s (aproximadamente - $\pi/2$) y esto corresponde a los momentos en que el robot se detiene para girar hacia la derecha.
- Presenta pulsos positivos que llegan a $+1.57$ rad/s (aproximadamente $\pi/2$) y esto corresponde a los momentos en que el robot se detiene para girar hacia la izquierda.